



Operating

Given a multi-org customer scenario, compare and contrast approaches for managing common release artifacts.

Develop your career with Salesforce Training and Certification Preparation

Table of Contents

-  [Managing Common Release Artifacts](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Describe the use of various approaches for managing common release artifacts in a multi-org architecture.
-  Recommend suitable strategies for managing common release artifacts for specific business requirements.



Introduction

In today's complex Salesforce environments, architects often face the challenge of **managing common release artifacts** across multiple Salesforce orgs. In a **multi-org architecture**, various approaches can be utilized for managing these shared components, which include custom fields, Apex classes, Lightning components, etc.

Salesforce architects should have the knowledge and tools to efficiently **synchronize updates, maintain consistency, and ensure the seamless operation of business processes** across different Salesforce orgs. Strategies for managing common release artifacts include **source-driven development, Metadata API-based deployments, package-based deployments**, and the use of **change sets**.



Managing Common Release Artifacts

Source-Driven Development

A version control system (VCS) such as Git should be utilized as the source of truth for all artifacts, ensuring consistency across orgs.

Package-Based Development

Managed, unmanaged, and unlocked packages can be created to deploy release artifacts or perform a metadata backup.



Metadata API

The `deploy()` and `retrieve()` calls of Metadata API can be used to retrieve and deploy release artifacts across Salesforce orgs.

Additional Strategies

Additional strategies for managing common release artifacts include creating a .zip file, using Salesforce CLI commands, evaluating third-party solutions, and using an RTM.

Managing Common Release Artifacts

Table of Contents



Multi-Org Architecture & Release Artifacts

When a company uses **multi-org architecture**, managing **common release artifacts** across multiple orgs presents certain challenges.

❖ ARTIFACTS

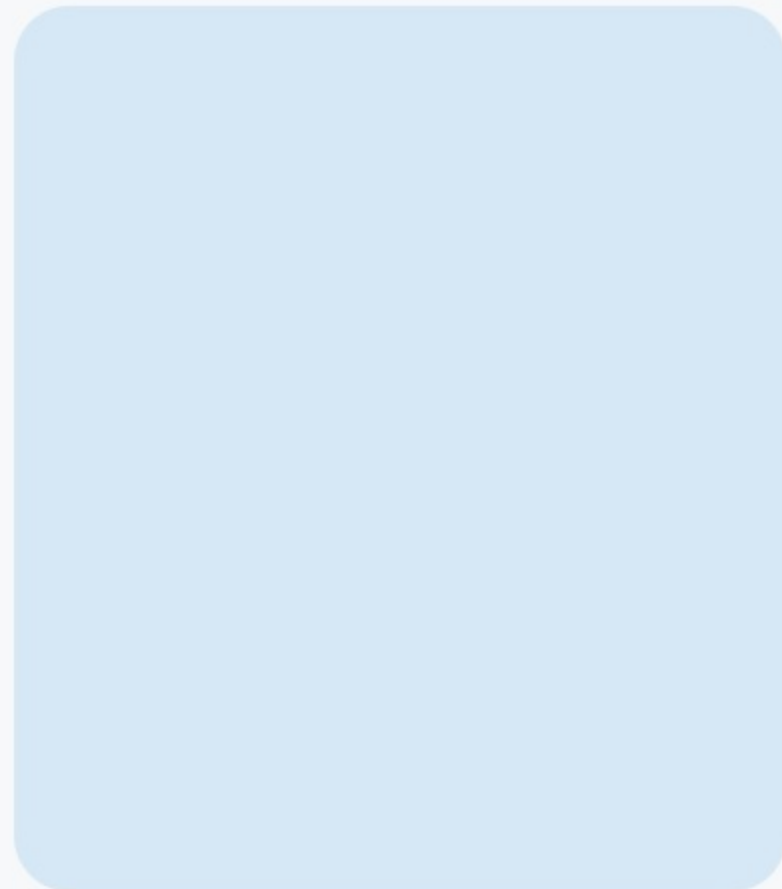
Common release artifacts that need to be synchronized across orgs include **code**, **configurations**, **metadata components**, and **data**.

❖ CHALLENGES

Managing common release artifacts across multiple orgs presents challenges in ensuring **consistency**, **governance**, and **efficiency**.

❖ APPROACHES

Managing release artifacts in a multi-org architecture requires understanding of **deployment methodologies**, **tools**, and **best practices**.



Source-Driven Development

A **version control system (VCS)** such as Git should be utilized as the **source of truth** for all artifacts, ensuring consistency across orgs.



A VCS should be at the **core** of the release management process. It allows for **version tracking**, **branching**, and **merging** of development work across different orgs.



Salesforce DX, **Git**, and the **Metadata API** can be used for maintaining consistency and governance across orgs, enabling **collaborative development** and helping to **resolve conflicts in shared artifacts**.



CI/CD tools like Jenkins, Circle CI, and Travis CI can be integrated with Salesforce DX to automate the **build, test, and deployment processes**, ensuring consistent deployments across orgs.

Metadata API

The **deploy()** and **retrieve()** calls in Metadata API can be used to **retrieve and deploy release artifacts** across Salesforce orgs.



Deployments based on Metadata API provide **direct control** over specific metadata components and are suitable for orgs with **heavily customized metadata**.



Salesforce CLI commands can be used to retrieve and deploy artifacts. The **project retrieve start** command is used to retrieve components. The **project deploy start** command is used to deploy components.



The **deploy()** and **retrieve()** calls are used to deploy and retrieve a **.zip file**. The .zip file contains a **project manifest (package.xml)** that lists what to retrieve or deploy and **XML components** organized into folders.

Package Based Development

Packages can be created to **deploy release artifacts** or perform a **metadata backup**.



A package encapsulates a **collection of metadata** for easy deployment and supports **versioning** for iterative development. It can be distributed to **other Salesforce users** and **orgs**.



Managed packages are less flexible for updates post-deployment. **Unmanaged packages** cannot be upgraded. **Unlocked packages** require the use of Salesforce DX.



An unmanaged package can be created for **metadata backup**, the **rollback** of metadata changes and **restoring** data efficiently in the event of deployment issues or system bugs often requires **more specialized solutions**.

Change Sets

Change Sets can be utilized to **deploy release artifacts** between **connected orgs**.



SMALL DEPLOYMENTS

Change Sets offer a **simple and intuitive** method for small deployments and require **no additional tools or setup**. They can be easily used by Salesforce administrators.



CONSIDERATIONS

Using Change Sets is limited to orgs **associated with the same production org** (e.g., sandbox to production). It **lacks the sophistication** needed for managing **complex dependencies**.

Additional Deployment Strategies for Release Artifacts

Additional strategies can be employed for **managing deployments**, **data integrity**, and **system integration** in Salesforce environments.



For the deployment of artifacts when there is no direct access to the org, a **.zip file containing a package.xml file** and **XML components** can be utilized.



Deployment of artifacts can be **validated** prior to actual deployment using the Salesforce CLI **project deploy validate** command. The **project deploy quick** command quickly deploys a validated deployment.



Third-party and AppExchange products can be evaluated to implement metadata rollback and data recovery to comply with industry regulations.



Comprehensive documentation, such as a **Requirements Traceability Matrix**, can be used to trace the development and purpose of code components.

Business Scenario

Table of Contents



Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Solutions, a global enterprise that offers various types of technology solutions, operates using multiple Salesforce orgs to cater to its diverse business units. Each org utilizes shared release artifacts, including custom fields, Apex classes, and Lightning components, vital for its CRM and project management processes. With the aim to streamline the development lifecycle and ensure consistency across all orgs, the company enlists the expertise of a Salesforce architect.

The architect is tasked with devising strategies to address specific requirements for managing common release artifacts efficiently while accommodating the unique needs of each org. The architect must explore approaches such as source-driven development, using Metadata API, package-based deployment, and using change sets.

Requirements & Solutions

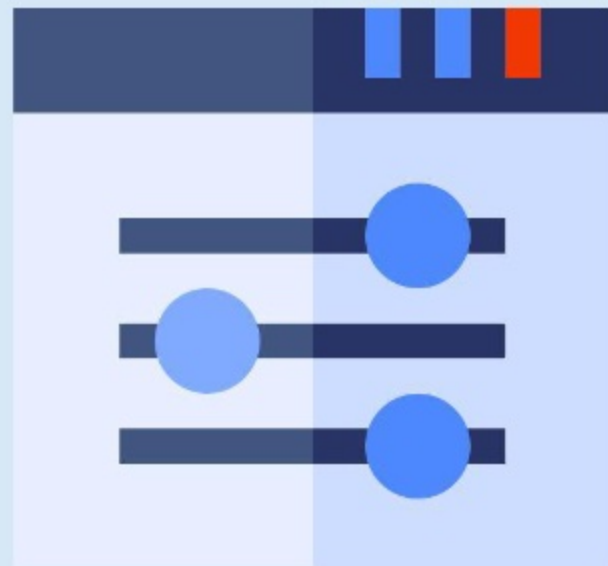
[Table of Contents](#) 

Requirement 1



SCENARIO

A Salesforce consultant has been working on a customization project, but due to security concerns, the company cannot provide the consultant with direct access to the production org that requires the new features. A dedicated release management team will be responsible for performing the deployment to production. The architect needs to recommend suitable approaches for the proper communication of metadata components to the release management team as well as their validation and final deployment to the production environment using Salesforce CLI.

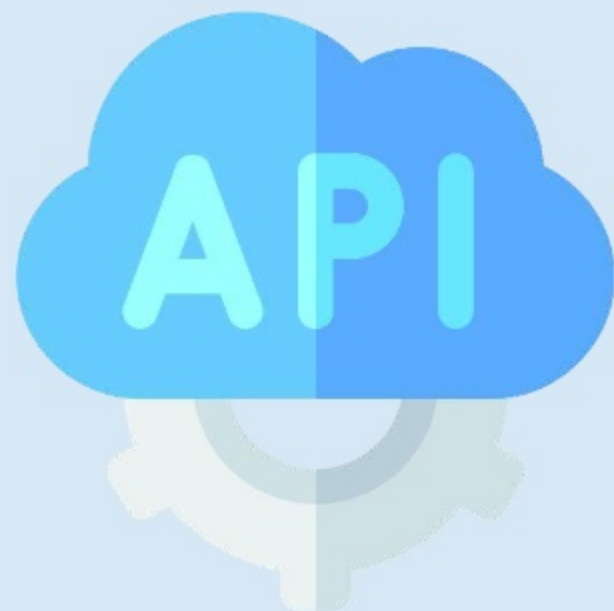


Solution 1



SOLUTION

The consultant can use the **retrieve()** call of **Metadata API** for this requirement. They should retrieve all the project metadata and create a **.zip file** containing a **project manifest (package.xml)** file with all the associated metadata and send it to the release management team. The project manifest lists what to retrieve or deploy. **One or more XML components** are organized into folders in the .zip file. After receiving the .zip file, the release management team can use the **project deploy validate** command to validate the metadata deployment. After successful validation, it can use the **project deploy quick** command for the actual deployment.



Requirement 2



SCENARIO

A developer would like to utilize an existing Apex class for a new functionality but is unsure of its purpose. The class was defined by a different developer for a project that went live two years ago in a different Salesforce org. Comprehensive documentation is still maintained for that project. The architect must suggest a suitable approach to determine the purpose of the class.



Solution 2



SOLUTION

Considering the project's documentation is still available, the developer can leverage the **Requirements Traceability Matrix (RTM)** to determine the purpose of the Apex class. It contains project specifications and artifacts, shows how requirements are related to the artifacts (such as an Apex class), and demonstrates that requirements have been met.

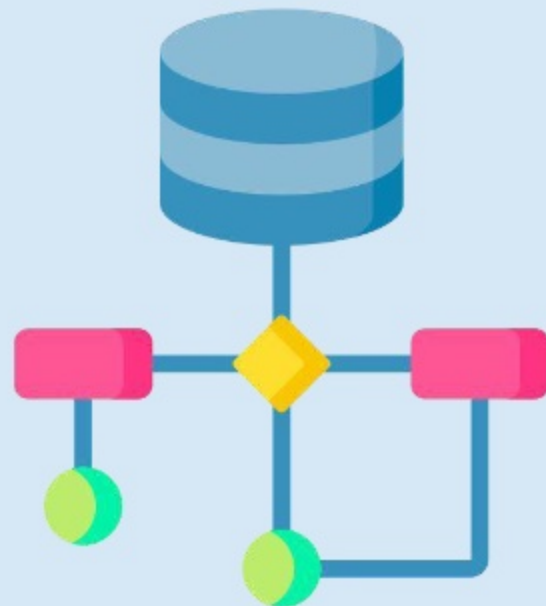


Requirement 3



SCENARIO

The company is rolling out new infrastructure and strategies based on Salesforce DevOps for its production org in Europe. The new infrastructure should facilitate the rollback of metadata after a deployment and data backup and restore to help recover from deployment issues, bugs, or outages in the downstream environments. To remain compliant with industry regulations, the company must implement an appropriate strategy to recover and roll back from issues as soon as possible.



Solution 3



SOLUTION

The company should **evaluate third-party and AppExchange solutions** for this use case. Although Salesforce offers the creation of a metadata backup using an **unmanaged package**, the rollback of metadata changes and restoring data efficiently in the event of deployment issues or system bugs often requires more specialized solutions.



Learn More

- [!\[\]\(6302aad5aed157b291fddf37b4870784_img.jpg\) What is a Package?](#)
- [!\[\]\(a9ca2c237943a6d0a9f22252f295b6f3_img.jpg\) Develop and Deploy Using Metadata API](#)
- [!\[\]\(9a01a64e0b4ff865df7d32ee7991fe8b_img.jpg\) Deploying and Retrieving Metadata with the Zip File](#)
- [!\[\]\(6aefe9a3d997eb8b55c40ecd5fa7053f_img.jpg\) Back Up Metadata to Protect and Restore Your Customizations](#)
- [!\[\]\(baa8f8ba8c970db55300f5bb45bb3460_img.jpg\) project Commands](#)

